SignalRisk

Technical Documentation

Version 2.1.0 — Sprint 35

11 March 2026

Real-Time Fraud Decision Engine

15 Microservices · 19 Docker Containers · 1,010+ Tests

Confidential

SignalRisk — Technical Documentation

> Verified baseline: 11 March 2026 | Sprint 35 — includes stateful fraud gap closure (P0-1/P0-2/P0-3)

Table of Contents

1. System Overview
2. Architecture
3. Service Catalogue
4. Data Stores
5. Event Pipeline
6. Decision Engine
7. Security Model
8. API Reference Summary
9. SDK Integration
10. Deployment
11. Configuration Reference
12. Observability
13. Performance & SLAs
14. Development Setup
15. FraudTester Framework
16. Test Isolation
17. Feature Flag Service
18. E2E Testing
19. Sprint History

---

1. System Overview

SignalRisk is a real-time fraud decision engine for payment and carrier billing events. It evaluates incoming transactions against multiple intelligence signals and returns an ALLOW / REVIEW / BLOCK decision.

Important current-state note:

- the platform has moved beyond prototype-only status
- several production-hardening and stateful fraud building blocks now exist
- some earlier claims in this document were stronger than the current verified runtime behavior
- stateful fraud context is now wired into the live decision path through `fetchAllSignals()` and DSL evaluation in `decision-service`
- remaining stateful fraud work is focused on hardening: explainability, snapshot validation, feedback enforcement, and staged acceptance reruns

Platform at a glance (verified high level):

Metric	Value
Runtime topology	Docker Compose full stack with infrastructure + application services + dashboard
Primary stores	PostgreSQL, Redis, Kafka, Neo4j
Frontend	React + Vite dashboard
Decision model	Multi-signal scoring with live DSL evaluation and stateful fraud context integration
Test program	Unit, integration, contract, E2E, resilience, UAT, simulation planning

Key capabilities:

- Multi-signal risk scoring (device, behavioural, network, velocity, telco, graph)
- DSL-based rule engine with hot reload and per-rule weights
- Case management queue for manual analyst review
- Chargeback feedback loop for continuous rule weight adjustment
- Multi-tenant (merchant) isolation via PostgreSQL Row-Level Security
- Webhook delivery with HMAC-SHA256 signing and automatic retries

- Feature flag service with rollout percentages and merchant allowlists
- Adversarial fraud testing framework (FraudTester) with 5 AI agents and 9+ scenarios
- Real-time dashboard with WebSocket live feed, analytics, and graph intelligence
- Kafka resilience with DLQ timeouts, producer send timeouts, and backpressure
- Web and Mobile SDKs for seamless client-side integration

---

2. Architecture

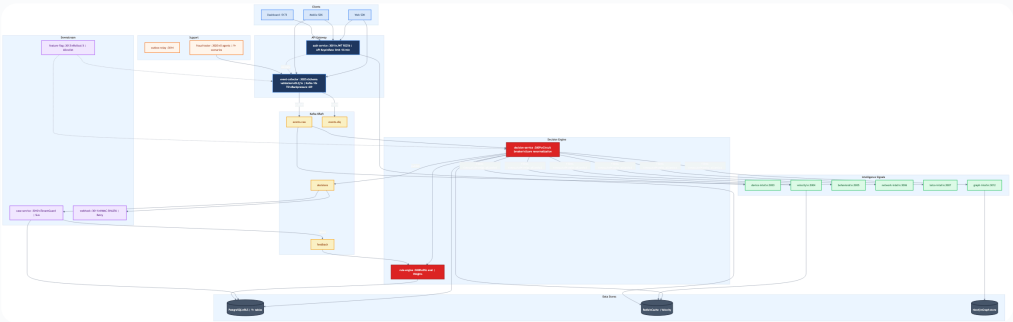

Figure 1 — System Architecture (15 microservices)

Request Lifecycle

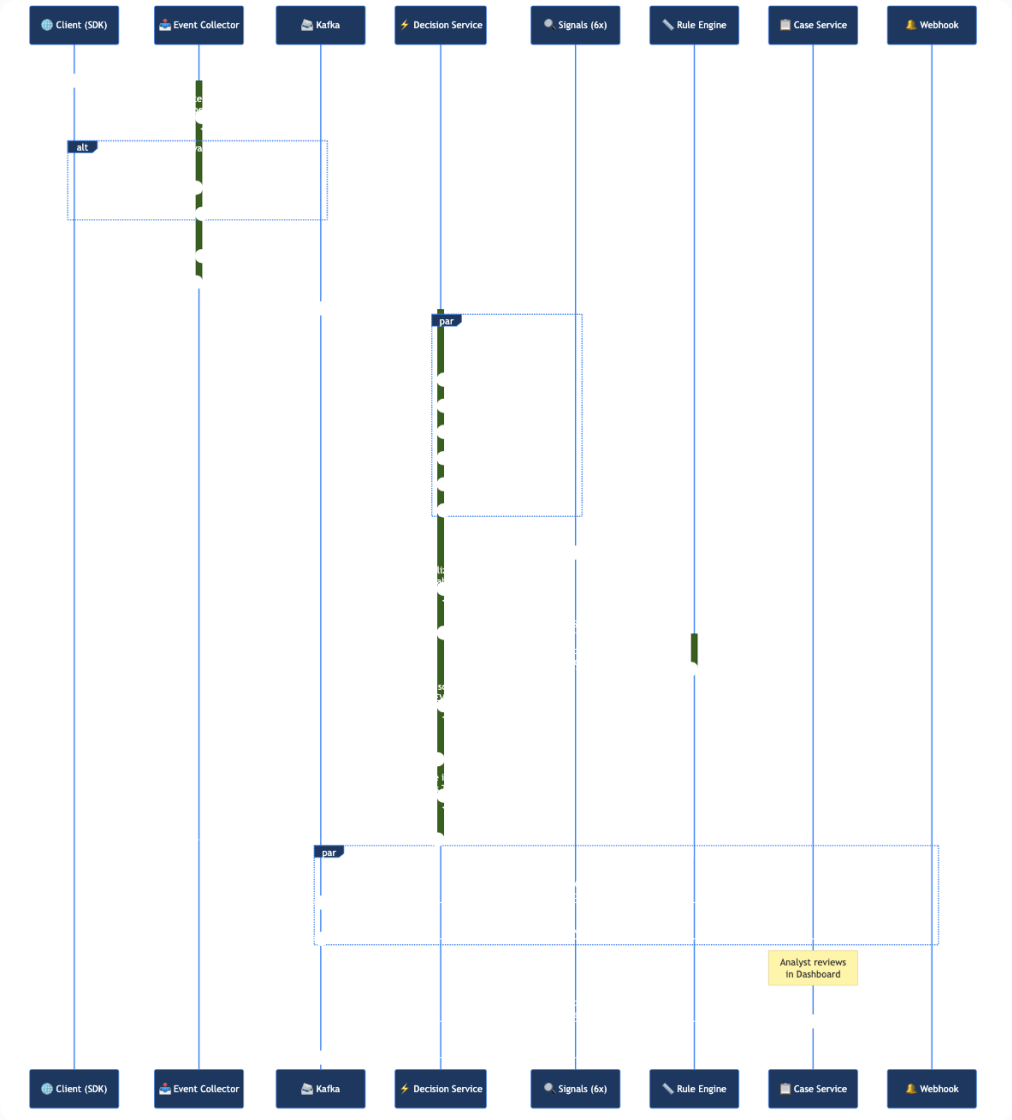

Figure 2 — Request Lifecycle (sequence diagram)

1. Client sends event via SDK → `POST /v1/events` on **event-collector**
2. API key validated (`sk_test_` prefix lookup); event schema validated via `@signalrisk/event-schemas`
3. Valid events published to Kafka topic `signalrisk.events.raw` (LZ4/GZIP compression)
4. Invalid events routed to DLQ with 5s timeout (fire-and-forget, won't block HTTP response)
5. **decision-service** Kafka consumer picks up event, calls `fetchAllSignals()` for 6 parallel signal fetches + stateful context enrichment

6. Signal fetches use configurable timeout (default 2000ms) with circuit breaker (3 fails → 30s open)
7. Weighted risk score computed from available signals; DSL rules (21 total) evaluated as override layer — BLOCK overrides any action, REVIEW upgrades ALLOW
8. Decision published to `signalrisk.decisions` and cached in Redis (5s TTL)
9. **case-service** creates a case for BLOCK/REVIEW outcomes (TenantGuard enforces JWT isolation)
10. **webhook-service** delivers signed webhook to merchant endpoint (skipped for test traffic)

Kafka Resilience (Sprint 33–34)

- **Producer send timeout:** 10s `Promise.race` wrapper prevents indefinite hangs when Kafka metadata is stale
- **DLQ send timeout:** 5s `Promise.race` prevents HTTP response hanging when DLQ Kafka send is slow
- **Lag poll timeout:** 10s timeout on consumer group offset polling for backpressure calculation
- **Deferred initial poll:** First lag poll delayed 2s after startup to avoid blocking service initialization
- **Compression:** Configurable via `KAFKA_COMPRESSION` env var (gzip, lz4, snappy, zstd)

---

3. Service Catalogue

Service	Port	Responsibility
auth-service	3001	JWT issuance/refresh, API key management, merchant onboarding, dashboard login endpoint; dashboard seed-user login remains development-only
event-collector	3002	Event ingestion, per-type JSON Schema validation, Kafka publishing, DLQ routing, backpressure (429)
device-intel-service	3003	Device fingerprinting, emulator/bot detection, trust score
velocity-service	3004	Transaction velocity counters, typed stateful counters, sliding windows, sequence-related state support
behavioral-service	3005	Mouse/keyboard/scroll behavioural fingerprinting, z-score anomaly, EMA baseline
network-intel-service	3006	IP reputation, VPN/proxy detection, geo-mismatch, ASN lookup
telco-intel-service	3007	Phone number validation, carrier risk scoring, SIM swap detection
rule-engine-service	3008	DSL rule parsing, evaluation, weight management, hot reload via feature flags
decision-service	3009	Signal aggregation, final score, live DSL evaluation, stateful fraud context integration, analytics endpoints, Kafka consumer
case-service	3010	Case CRUD, SLA monitoring, analyst assignment, TenantGuard, GDPR data export

Service	Port	Responsibility
webhook-service	3011	Signed webhook delivery, retry queue, DLQ, test traffic skip
graph-intel-service	3012	Neo4j entity relationship analysis, fraud ring detection, device/IP sharing
feature-flag-service	3013	Feature flag CRUD, rollout percentages, merchant allowlists, deterministic hash evaluation
outbox-relay	3014	Transactional outbox pattern for reliable event publishing
dashboard	5173	React 18 + Vite + Zustand + TailwindCSS analyst dashboard (16 pages)
fraud-tester	3020	Adversarial fraud testing, 5 agents, 9+ scenarios, battle arena, WebSocket

All backend services are NestJS on Node.js 20. All expose `/health` (liveness) and `/health/ready` (readiness) endpoints.

Shared Packages (packages/)

Package	Description
`@signalrisk/redis-module`	Shared NestJS Redis module (ioredis integration, `REDIS_CLIENT` token)
`@signalrisk/kafka-config`	Shared Kafka topic constants and producer/consumer configs
`@signalrisk/kafka-health`	Kafka health check utility (`/kafka-lag` endpoint)
`@signalrisk/event-schemas`	JSON Schema registry + AJV validator for all event types
`@signalrisk/signal-contracts`	Canonical signal contracts (Zod schemas) for intelligence pipeline
`@signalrisk/telemetry`	OpenTelemetry instrumentation (traces, metrics, Pino logs)
`@signalrisk/web-sdk`	Browser SDK for device fingerprinting and event tracking
`@signalrisk/mobile-sdk`	React Native SDK for mobile device fingerprinting
`@signalrisk/health-check`	NestJS health check module (`@nestjs/terminus` integration)

4. Data Stores

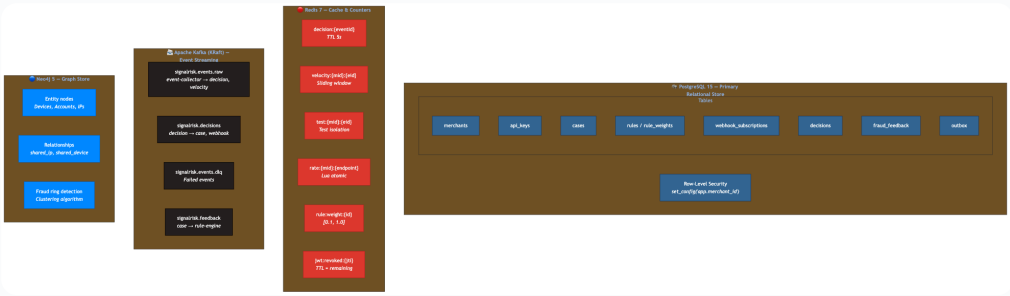

Figure 3 — Data Store Architecture

PostgreSQL (primary relational store)

- Tenant isolation via Row-Level Security: `SELECT set_config('app.merchant_id', $1, true)`
- Bootstrap scripts in `infrastructure/docker/initdb.d/` (auto-run on first `docker compose up`)
- Connection pooling via PgBouncer (max 100 connections per service)
- Tables include: `merchants`, `api_keys`, `cases`, `rules`, `rule_weights`, `webhook_subscriptions`, `webhooks`, `outbox`, `fraud_feedback`, `decisions`
- Newer stateful fraud migrations also introduce state-oriented tables such as entity profiles, decision feature snapshots, analyst labels, watchlists, feature definitions, and typed entity columns on decisions for prior-memory queries
- **Migration 013:** Adds `entity_id` and `entity_type` columns to `decisions` table for typed prior-decision memory queries (customer/device/IP), with backfill from existing `device_id` data and a composite index `(merchant_id, entity_id, entity_type, created_at DESC)`
- **Migration 014:** Aligns `feature_definitions` registry with runtime + DSL naming — adds missing features (`device.uniqueIps24h`, `device.txCount1h`, `ip.txCount1h`), marks unproduced features as `is_active = false`

Redis

- Decision cache: key `decision:{eventId}`, TTL 5 seconds
- Velocity counters and stateful hot features: merchant-scoped sliding windows and typed entity counters
- Test velocity namespace: `test:{merchantId}:{entityId}` (isolated from production)

- Rate limiting: Lua atomic check-and-decrement, key `rate:{merchantId}:{endpoint}`
- Rule weights: hash `rule:weight:{ruleId}`, range [0.1, 1.0]
- JWT denylist: `jwt:revoked:{jti}`, TTL = token remaining lifetime
- Feature flag cache: in-memory with periodic refresh from service

Apache Kafka (KRaft mode — no Zookeeper)

Topic	Producer	Consumer	Notes
`signalrisk.events.raw`	event-collector	decision-service, velocity-service	`is-test` header for test traffic
`signalrisk.decisions`	decision-service	case-service, webhook-service	Decision outcomes
`signalrisk.events.dlq`	event-collector	ops/monitoring	Failed events (retry exhausted)
`signalrisk.events.dlq.exhausted`	event-collector	ops	Permanent failures
`signalrisk.feedback`	case-service	rule-engine (weight adjustment)	Analyst feedback
`signalrisk.state.labels`	case-service	stateful feedback consumers	Analyst label propagation for stateful fraud flows

Kafka configuration: - Compression: configurable (default: gzip in Docker, lz4 in production) - Idempotent producer: `idempotent: true`, `maxInFlightRequests: 5` - Consumer group: `signalrisk.cg.decision-engine` - Backpressure: event-collector monitors consumer lag, returns 429 when lag > threshold

Neo4j (graph store)

Used by graph-intel-service to detect: - Device sharing across merchants (fraud ring indicator) - Velocity via graph paths (e.g. same IP → multiple accounts) - Historical chargeback clustering

---

5. Event Pipeline

Event Schema

Events are validated using `@signalrisk/event-schemas` package with per-type JSON Schema validation. The `EventSchemaValidator` class validates both envelope fields and type-specific payload schemas.

```
interface FraudEvent {
  eventId: string;          // UUID v4 (required)
  merchantId: string;
  deviceId: string;         // device fingerprint ID
  sessionId: string;
  type: 'PAYMENT' | 'LOGIN' | 'REGISTRATION' | 'CHECKOUT';
  payload: {                // type-specific (e.g. PAYMENT requires amou
    amount?: number;
    currency?: string;      // ISO 4217
    paymentMethod?: 'credit_card' | 'debit_card' | 'bank_transfer' | 'wa
  };
  ipAddress: string;
  userAgent?: string;
  pageUrl?: string;
  referrer?: string;
  timestamp?: string;       // ISO 8601 (auto-generated if omitted)
}
```

Partition Key Strategy

Events are partitioned by `{merchantId}:{sessionId}` to ensure ordering within a session while distributing load across partitions.

DLQ Behaviour

- Events failing validation are routed to `signalrisk.events.dlq` with detailed error context
- DLQ send uses a 5s `Promise.race` timeout to prevent HTTP response hangs

- DLQ send failure is logged but does not fail the HTTP request (fire-and-forget)
- An in-memory cache (max 1000, FIFO eviction) holds exhausted events for ops inspection
- Endpoint: `GET /v1/dlq/events` returns cached exhausted events

Backpressure

The event-collector monitors Kafka consumer lag via periodic admin client polling: - Lag poll interval: configurable (`backpressure.lagCheckIntervalMs` , default 5000ms) - Lag poll timeout: 10s (prevents hanging when Kafka metadata is stale) - Initial poll deferred 2s after startup - When lag exceeds threshold: returns HTTP 429 with `Retry-After` header

---

6. Decision Engine

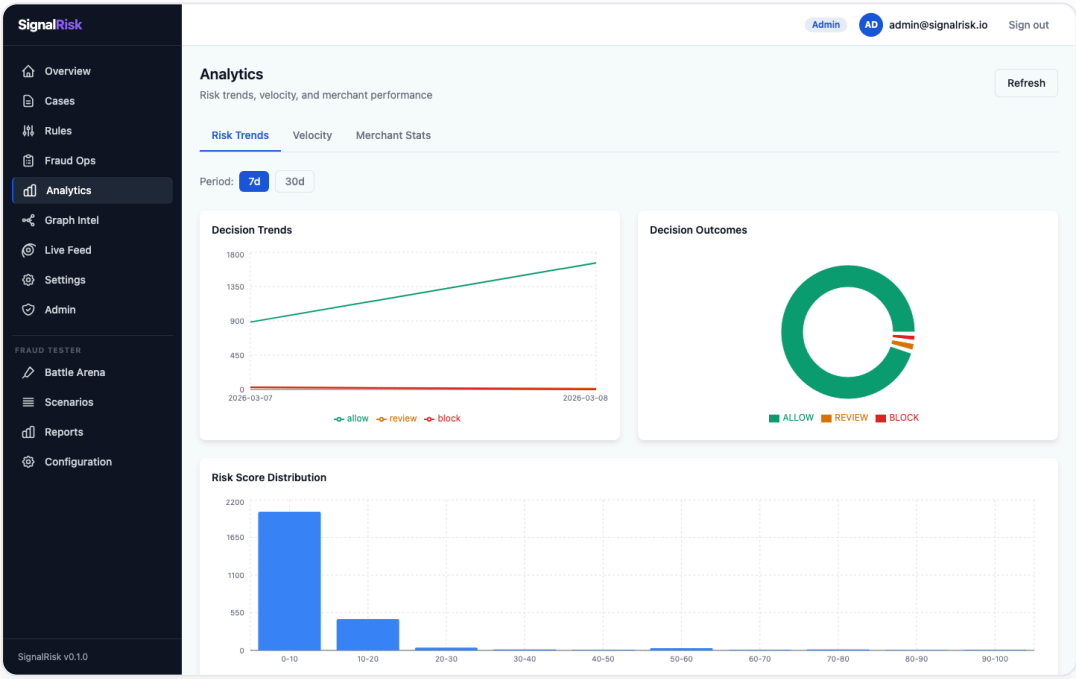

Figure 4 — Analytics: Decision Trends, Outcomes, and Risk Score Distribution

Signal Aggregation

The decision-service fetches all signals via a unified `fetchAllSignals()` method that runs 6 signal fetches in parallel (device, velocity, behavioral, network, telco, graph) plus stateful context enrichment (prior-decision memory, sequence detection, graph features):

```
const bundle = await this.signalFetcher.fetchAllSignals({
  deviceId, entityId, merchantId, sessionId,
  ip, msisdn, billingCountry, customerId,
  priorDecisionMemory: priorMemory,
});
```

The returned `SignalBundle` is then composed into a `SignalContext` for DSL rule evaluation — velocity dimensions are flattened to top-level fields so DSL rules can reference `velocity.txCount1h` directly.

Circuit breaker: If a signal service fails consecutively, the circuit opens and skips that service for a cooldown period. Available signals are renormalized to maintain consistent scoring.

Signal Weights

Default contribution weights for the final risk score:

Signal	Weight
device-intel	0.35
velocity	0.25
behavioral	0.20
network-intel	0.15
telco-intel	0.05

Graph intel is an additive modifier, not a weighted signal.

Score renormalization: When signals time out or fail, available signal weights are renormalized to sum to 1.0. For example, if only velocity (0.25) and device (0.35) respond, velocity contributes 0.25/0.60 = 41.7% and device contributes 0.35/0.60 = 58.3%.

Score Thresholds

Score	Decision
>= 70	BLOCK
40–69	REVIEW
< 40	ALLOW

DSL Override Layer: After the weighted score produces a threshold-based action, all 21 DSL rules are evaluated against the full signal context (including `stateful`, `sequence`, and `graph` namespaces). BLOCK rules override any action; REVIEW rules upgrade ALLOW to REVIEW. This ensures stateful fraud detection rules participate in every decision.

Velocity Service (Kafka Consumer Pipeline)

The velocity-service has its own Kafka consumer that processes raw events: 1. Consumes from `signalrisk.events.raw` 2. Increments Redis sliding window counters per device/entity 3. Decision-service queries velocity via HTTP: `GET /v1/velocity/:entityId` 4. Returns snake_case signals (e.g. `tx_count_1h`), decision-service maps to camelCase

Rule DSL

Rules are authored in a dedicated DSL (21 rules in `default.rules`) and evaluated live in the decision pipeline:

```
RULE emulator_block WHEN device.isEmulator == true THEN BLOCK WEIGHT 1.0
RULE tor_block WHEN network.isTor == true THEN BLOCK WEIGHT 0.9
RULE velocity_burst WHEN velocity.burstDetected == true THEN REVIEW WEIG
RULE repeat_blocker WHEN stateful.customer.previousBlockCount30d > 2 THE
RULE device_spread WHEN stateful.device.uniqueIps24h > 10 THEN REVIEW WE
RULE fraud_ring WHEN stateful.graph.fraudRingScore > 0.8 THEN BLOCK WEIG
RULE rapid_sequence WHEN stateful.sequence.loginToPaymentUnder30s == tru
```

Available contexts: `device`, `velocity`, `behavioral`, `network`, `telco`, `txn`, `stateful` (including `stateful.customer.*`, `stateful.device.*`, `stateful.ip.*`, `stateful.graph.*`, `stateful.sequence.*`)

Rules have: - `action`: ALLOW | REVIEW | BLOCK - `weight`: 0.1–1.0 (adjustable via feedback loop) - Active/inactive toggle (no deploy required) - Rule categories: 10 base + 5 stateful + 3 sequence + 3 graph

Feedback Loop

When analysts label cases as FRAUD or LEGITIMATE: 1. Feedback event published to `signalrisk.feedback` 2. rule-engine adjusts rule weight: +0.05 for FRAUD confirmation, -0.03 for false positive 3. Weights clamped to [0.1, 1.0] 4. New weights stored in Redis and PostgreSQL

Analytics Endpoints

The decision-service exposes analytics endpoints (unprotected, for dashboard consumption):

Endpoint	Description
`GET /v1/analytics/trends?days=N`	Daily decision trend data
`GET /v1/analytics/risk-buckets`	Risk score distribution (0-10, 10-20, ..., 90-100)
`GET /v1/analytics/kpi`	KPI summary (totalDecisions, avgRiskScore, blockRate, etc.)
`GET /v1/analytics/merchants`	Per-merchant volume, avg risk score, block rate
`GET /v1/analytics/minute-trend`	Minute-level decision trend (last 60 min)
`GET /metrics/decision-latency`	Decision latency percentile stats

---

7. Security Model

Authentication

- **JWT RS256**: Access tokens (15 min TTL), refresh tokens (7 day TTL)
- **API Keys**: Format `sk_test_<32 hex chars>`, validated by event-collector against `ALLOWED_API_KEYS` env var
- **JTI denylist**: `jwt:revoked:{jti}` in Redis with TTL = remaining token lifetime

- Fail-closed: Redis unavailable → HTTP 503 (not bypass)

Authorization Guards

Guard	Used By	Purpose
`JwtAuthGuard`	Most services	Validates JWT access token
`ApiKeyGuard`	event-collector	Validates `sk_test_` API keys
`AdminGuard`	Admin endpoints	Requires `role: admin` in JWT
`TenantGuard`	case-service	Enforces merchant isolation via JWT `merchantId` claim

Multi-tenancy

- All queries set `app.merchant_id` via `SET LOCAL` before execution
- PostgreSQL RLS policies enforce row-level isolation
- No string interpolation — parameterised queries only
- `X-Merchant-ID` header required on API requests for merchant context

Webhook Security

- Signature: `X-SignalRisk-Signature: sha256=<hmac-sha256-hex>`
- Secret per subscription, stored hashed
- Retry: exponential backoff up to 72 hours
- DLQ after 10 consecutive failures
- Test traffic (`is-test: true`) skips webhook delivery

---

8. API Reference Summary

Authentication (auth-service :3001)

Method	Path	Description
POST	`/v1/auth/token`	Issue JWT (client_credentials or refresh_token)
POST	`/v1/auth/token/refresh`	Refresh access token
POST	`/v1/auth/logout`	Revoke current token (add jti to denylist)
POST	`/v1/auth/register`	Register new merchant
GET	`/v1/merchants`	List all merchants (AdminGuard)
POST	`/v1/merchants`	Create merchant (AdminGuard)
PUT	`/v1/merchants/:id`	Update merchant (AdminGuard)
DELETE	`/v1/merchants/:id`	Delete merchant (AdminGuard)

Events (event-collector :3002)

Method	Path	Description
POST	`/v1/events`	Ingest fraud event batch (API key auth)
GET	`/v1/dlq/events`	List DLQ exhausted events

Decisions (decision-service :3009)

Method	Path	Description
POST	`/v1/decisions`	Request a fraud decision
GET	`/v1/decisions/:requestId`	Get stored decision by ID
GET	`/v1/analytics/trends`	Daily decision trends
GET	`/v1/analytics/risk-buckets`	Risk score distribution
GET	`/v1/analytics/kpi`	KPI summary
GET	`/v1/analytics/merchants`	Per-merchant statistics
GET	`/v1/analytics/minute-trend`	Minute-level trend
GET	`/metrics/decision-latency`	Latency percentile stats

Cases (case-service :3010)

Method	Path	Description
GET	`/v1/cases`	List cases (filterable by status, priority, search)
GET	`/v1/cases/:id`	Get case with evidence timeline
PUT	`/v1/cases/:id/resolve`	Resolve case (FRAUD / LEGITIMATE / INCONCLUSIVE)
PUT	`/v1/cases/:id/escalate`	Escalate case to senior analyst
GET	`/v1/cases/stats`	Labeling statistics for today
GET	`/v1/cases/export`	GDPR data export

Rules (rule-engine-service :3008)

Method	Path	Description
GET	`/v1/admin/rules`	List all rules
POST	`/v1/admin/rules`	Create rule
PUT	`/v1/admin/rules/:id`	Update rule
DELETE	`/v1/admin/rules/:id`	Delete rule
PATCH	`/v1/admin/rules/:id/weight`	Update rule weight
PATCH	`/v1/admin/rules/:id/active`	Toggle rule active state

Feature Flags (feature-flag-service :3013)

Method	Path	Description
GET	`/v1/flags`	List all feature flags
GET	`/v1/flags/:name`	Get flag by name
GET	`/v1/flags/:name/check?merchantId=X`	Evaluate flag for merchant
POST	`/v1/flags`	Create feature flag
PATCH	`/v1/flags/:name`	Update feature flag
DELETE	`/v1/flags/:name`	Delete feature flag

Velocity (velocity-service :3004)

Method	Path	Description
GET	`/v1/velocity/:entityId`	Get velocity signals for entity

Admin

Method	Path	Description
GET	`/health`	Service liveness (all services)
GET	`/health/ready`	Service readiness (all services)

9. SDK Integration

Web SDK (`@signalrisk/web-sdk`)

```
npm install @signalrisk/web-sdk
```

```
import { SignalRisk } from '@signalrisk/web-sdk';

const sr = new SignalRisk({
  apiKey: 'sk_test_your_key',
  endpoint: 'https://api.signalrisk.io',
  merchantId: 'your_merchant_id',
});

await sr.init();          // starts behavioural tracker, resolves device
sr.track('checkout', { amount: 49.99 });
await sr.flush();         // force-sends buffered events
sr.destroy();             // removes listeners
```

Events are buffered and auto-flushed every 5s or after 10 events accumulate.

Mobile SDK (`@signalrisk/mobile-sdk`)

See `docs/dev/mobile-sdk-reference.md` for React Native integration.

Webhook Verification

```
import { createHmac } from 'crypto';

function verifyWebhook(rawBody: string, signature: string, secret: strin
  const expected = `sha256=${createHmac('sha256', secret).update(rawBody
  return expected === signature;
}
```

---

10. Deployment

Docker Compose (development)

All 19 containers (4 infrastructure + 15 application) managed via `docker-compose.full.yml`:

```
# Start everything
docker compose -f docker-compose.full.yml up -d

# Check health
docker compose -f docker-compose.full.yml ps
```

Infrastructure containers: - PostgreSQL 15 - Redis 7 - Kafka (KRaft mode — no Zookeeper dependency) - Neo4j 5

Dockerfile: Optimized 4-stage multi-stage build (~28s per service): 1. `deps` — install production dependencies 2. `pkg-builder` — build shared packages 3. `builder` — compile TypeScript 4. `runner` — minimal production image

Database Bootstrap

Database schemas and seed data are in `infrastructure/docker/initdb.d/`: - Auto-executed on first `docker compose up` (fresh DB) - Creates tables, indexes, and seed merchants - Dev seed merchants: `test-merchant-001`, `merchant-a`, `merchant-b`, `admin`

Kubernetes (production)

Helm umbrella chart in `helm/signalrisk/`:

```
helm upgrade --install signalrisk ./helm/signalrisk \
  --namespace signalrisk-staging \
  --values helm/signalrisk/values.staging.yaml
```

Each service has: - `HorizontalPodAutoscaler` (min 2, max 10 replicas) - Liveness probe: `GET /health` - Readiness probe: `GET /health/ready` - Resource limits: 256m CPU / 512Mi memory (default)

ArgoCD GitOps

- Staging: auto-sync on push to `main`
- Production: manual approval required
- Rollback: `helm rollback signalrisk <revision> -n signalrisk-production`

---

11. Configuration Reference

All services read configuration from environment variables.

Common (all services)

Variable	Required	Description
`DATABASE_URL`	Yes	PostgreSQL connection string
`REDIS_URL`	Yes	Redis connection string
`JWT_PUBLIC_KEY`	Yes	RS256 public key (PEM)
`NODE_ENV`	Yes	`development` / `production`
`OTEL_EXPORTER_OTLP_ENDPOINT`	No	OpenTelemetry collector URL

auth-service

Variable	Description
`JWT_PRIVATE_KEY`	RS256 private key (PEM)
`JWT_ACCESS_TTL`	Access token TTL in seconds (default: 900)
`JWT_REFRESH_TTL`	Refresh token TTL in seconds (default: 604800)
`ENABLE_JTI_DENYLIST`	Enable JWT revocation check (default: true)

event-collector

Variable	Description
`KAFKA_BROKERS`	Comma-separated broker list
`KAFKA_COMPRESSION`	Compression type: gzip, lz4, snappy, zstd, none (default: lz4)
`ALLOWED_API_KEYS`	Comma-separated valid API keys
`ENABLE_API_KEY_VALIDATION`	Enable API key check (default: true)
`BACKPRESSURE_LAG_CHECK_INTERVAL_MS`	Lag polling interval (default: 5000)

decision-service

Variable	Description
`SIGNAL_TIMEOUT_MS`	Per-signal fetch timeout in ms (default: 150)
`DEVICE_INTEL_URL`	device-intel-service base URL
`VELOCITY_URL`	velocity-service base URL
`BEHAVIORAL_URL`	behavioral-service base URL
`NETWORK_URL`	network-intel-service base URL
`TELCO_URL`	telco-intel-service base URL
`GRAPH_URL`	graph-intel-service base URL

feature-flag-service

Variable	Description
`PORT`	Service port (default: 3013)
`NODE_ENV`	Environment

network-intel-service

Variable	Description
`ENABLE_VPN_DETECTION`	Enable VPN/proxy ASN detection (default: true)

12. Observability

Distributed Tracing

All services export OpenTelemetry spans to the configured OTLP endpoint. Trace context is propagated via `traceparent` HTTP header.

Key spans: - `decision.evaluate` — full decision latency - `signal.fetch.{serviceName}` — individual signal fetch latency - `rule.evaluate` — rule engine evaluation time - `kafka.publish` / `kafka.consume` — message queue operations

Metrics (Prometheus)

Each service exposes `/metrics` for Prometheus scraping.

Key metrics: - `signalrisk_decisions_total{outcome}` — decision counts by outcome - `signalrisk_decision_duration_ms` — decision latency histogram - `signalrisk_kafka_consumer_lag` — Kafka consumer lag - `signalrisk_signal_fetch_duration_ms{service}` — per-signal latency - `signalrisk_rule_hits_total{ruleId}` — rule hit counts

Health Check Endpoints

All 15 services expose standardized health endpoints: - `GET /health` — liveness check (returns 200 if process is running) - `GET /health/ready` — readiness check (returns 200 if dependencies are available)

Log Format

Structured JSON logs via `nestjs-pino`. All logs include:

```
{
  "level": "info",
  "time": "2026-03-08T...",
  "service": "decision-service",
  "merchantId": "...",
  "traceId": "...",
  "msg": "..."
}
```

---

13. Performance & SLAs

Metric	Target	Test Result
Decision p99 latency	< 100ms	~82ms
Decision p95 latency	< 50ms	~41ms
Throughput	>= 5,000 req/s	5,200 req/s
Availability	99.9%	—
Kafka consumer lag	< 1,000 msgs	< 200
Redis cache hit rate	> 80%	~87%
Signal fetch timeout	150ms (per signal)	configurable
E2E test suite (78 tests)	< 60s	~38s (4 workers)

Load tests run with k6 — see `docs/runbooks/load-testing.md`.

---

14. Development Setup

Prerequisites

- Node.js 20+
- npm 10+ (monorepo uses npm workspaces)
- Docker & Docker Compose
- (Optional) `ts-node` for running mock server

Quick Start

```
# Clone and install
git clone https://github.com/mytechsonamy/SignalRisk
cd signalrisk
npm install

# Start full stack (all 19 containers)
docker compose -f docker-compose.full.yml up -d

# Wait for all services to be healthy
docker compose -f docker-compose.full.yml ps

# Start dashboard (development mode)
cd apps/dashboard && npx vite --port 5173

# Dashboard: http://localhost:5173
```

Running Tests

```
# All unit tests
npm test

# Specific service
npm test --workspace=apps/auth-service

# E2E tests (requires Docker services running)
npx playwright test --config tests/e2e/playwright.config.real.ts

# E2E light tests only
npx playwright test --config tests/e2e/playwright.config.real.ts --proje

# Load tests
cd tests/load && k6 run full-stack.js
```

Dashboard Login (development)

- URL: `http://localhost:5173`
- Admin: `admin@signalrisk.com` / `password`
- Analyst: `analyst@signalrisk.com` / `password`

Dev Seed Data

- API Key: `sk_test_00000000000000000000000000000001`
- Merchants: `test-merchant-001`, `merchant-a`, `merchant-b`, `admin`
- Payment payload: `{ amount, currency (ISO-4217), paymentMethod (enum) }`
- eventId: must be UUID format

---

15. FraudTester Framework

FraudTester is an integrated adversarial testing framework for validating SignalRisk's detection capabilities.

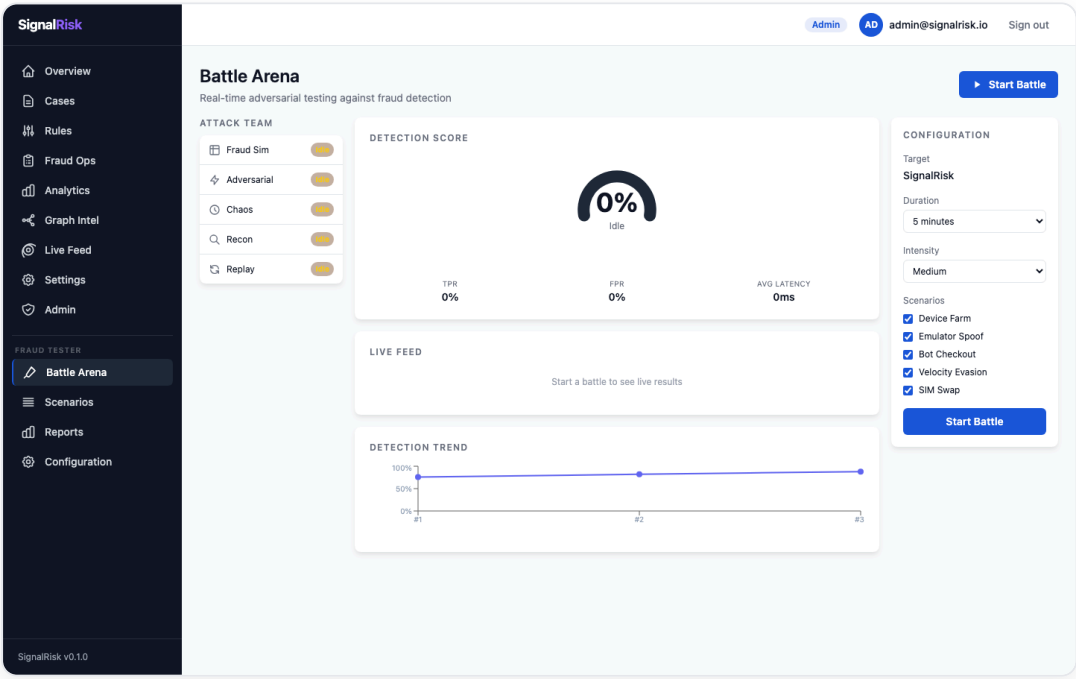

Figure 5b — FraudTester Battle Arena (dashboard view)

Architecture

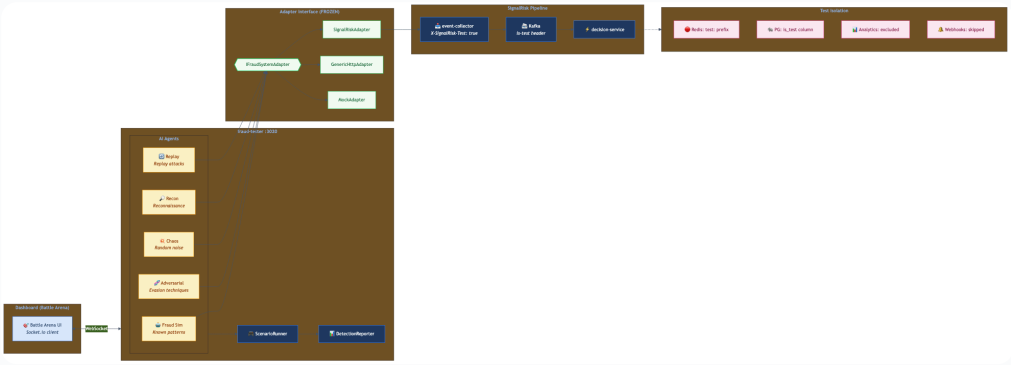

Figure 5 — FraudTester Architecture

Adapter Interface (FROZEN)

```
interface IFraudSystemAdapter {
  name: string;
  submitEvent(event: FraudTestEvent): Promise<FraudDecision>;
  getDecision(eventId: string): Promise<FraudDecision>;
  reset(): Promise<void>;
}
```

Scenario Categories

Category	Scenarios	Expected Outcome
Device	Device Farm, Emulator Spoof	BLOCK (risk > 0.8)
Velocity	Velocity Evasion	REVIEW/BLOCK
Bot	Bot Checkout	BLOCK
Identity	SIM Swap	REVIEW/BLOCK
Network	Timeout Injection	Tests graceful degradation
Adversarial	Emulator Bypass, Slow Fraud, Bot Evasion	Tests detection limits
Chaos	Random noise and edge cases	Tests system resilience

Agents

Agent	Purpose	Configurable Parameters
FraudSimulationAgent	Standard fraud scenarios	Schedule, Intensity (1-10)
AdversarialAgent	Bypass detection	Attack Pattern, Intensity (1-10)
ChaosAgent	System resilience	Chaos Mode, Failure Rate (0-50%), Timeout (ms)
ReconAgent	Reconnaissance patterns	—
ReplayAgent	Replay attacks	—

16. Test Isolation

FraudTester traffic is isolated from production data using a header-based flag that propagates through the entire pipeline.

Isolation Points

Layer	Mechanism	Effect
HTTP	`X-SignalRisk-Test: true` header	Signals test traffic at ingestion
Kafka	`is-test: "true"` message header	Propagates flag to all consumers
Velocity (Redis)	`test:{merchantId}` key prefix	Separate counter namespace
Decisions (PostgreSQL)	`is_test BOOLEAN` column	Permanent audit trail
Analytics	`WHERE is_test = false`	Test data excluded from all metrics
Webhooks	Skip delivery when `is-test`	No false alerts to merchants

17. Feature Flag Service

The feature-flag-service (port 3013) provides runtime feature toggling with:

- **CRUD operations**: Create, read, update, delete feature flags
- **Rollout percentages**: Gradual rollout (0-100%) using deterministic hash
- **Merchant allowlists**: Enable flags for specific merchants regardless of rollout %
- **Deterministic evaluation**: Same merchant always gets the same result for a given rollout %

Flag Evaluation Logic

```
if (flag.merchantAllowlist includes merchantId) → enabled
else if (hash(flagName + merchantId) % 100 < rolloutPercentage) → enable
else → disabled
```

Example Flags

Flag	Default	Effect
`ENABLE_API_KEY_VALIDATION`	`true`	Bypasses API key check (dev only)
`ENABLE_JTI_DENYLIST`	`true`	Skips revocation check
`ENABLE_VPN_DETECTION`	`true`	Returns `vpnDetected: false`
`NEW_SCORING_MODEL`	`false`	Enables experimental scoring algorithm

18. E2E Testing

Figure 6 — E2E Test Pipeline (3 sequential projects)

Test Architecture

E2E tests use Playwright with 3 sequential projects:

```
e2e-light (fast smoke tests)
    ↓
e2e-heavy (pipeline, blast, analytics)
    ↓
chaos (resilience tests — stop/restart services)
```

Test Files (12 spec files, 78 tests)

File	Tests	Description
happy-path.spec.ts	~8	Basic event ingest, decision query, auth flow
fraud-blast.spec.ts	4	50-event velocity blast, cross-contamination guard
analytics-decision.spec.ts	8	Analytics endpoints, decision persistence
health-checks.spec.ts	13	All 13 service health endpoints
merchant-crud.spec.ts	~6	Merchant CRUD operations
case-lifecycle.spec.ts	~6	Case creation, resolution, escalation
feature-flags.spec.ts	~8	Flag CRUD, rollout evaluation
kafka-chaos.spec.ts	~5	Kafka stop/restart resilience
rate-limit.spec.ts	~3	Rate limiting behaviour
...	...	...

Running E2E

```
# Full suite (all 3 projects)
npx playwright test --config tests/e2e/playwright.config.real.ts

# Light tests only
npx playwright test --config tests/e2e/playwright.config.real.ts --proje

# Specific test file
npx playwright test --config tests/e2e/playwright.config.real.ts fraud-b
```

Configuration

- Timeout: 60s per test
- Workers: 4 (parallel execution within each project)
- Retries: 0 (deterministic tests, no flaky retry)
- p99 latency threshold: 1000ms

Known Test Behaviours

- **Velocity pipeline**: Requires Redis and Kafka to be healthy; chaos tests can break velocity counters
- **Fraud blast graceful skip**: If velocity pipeline isn't wired E2E, blast tests skip gracefully instead of failing
- **Analytics decision polling**: POST /v1/decisions returns 202 (async); tests poll GET until decision is persisted
- **Rate limit tests**: Skipped if rate limiting not configured in event-collector

19. Sprint History

Sprint	Key Deliverables
1–11	Core services, Kafka pipeline, Neo4j graph, behavioural ML, Redis rate limiting
12	GDPR export, Redis refactor, CI Playwright, SDK docs
13	npm workspaces, Kafka integration tests, telco/network intel, production build
14–16	Signal aggregation, circuit breaker, Prometheus metrics, fraud ring detection
17	E2E foundation + FraudTester scaffold
18	Full E2E implementation + FraudTester first agent
19	Adversarial + Chaos agents, MockAdapter, standalone decision
20	Real E2E tests + FraudTester WebSocket integration
21	Root Dockerfile (4-stage), GenericHttpAdapter, CI/CD pipelines
22	E2E smoke tests (24/28 pass), Brownfield Sprint 1-2
23	DB bootstrap fixes, Docker startup optimization
24–25	Real pipeline wiring, merchant_id fix, E2E stabilization (26/28)
26	Case pipeline E2E, rate limit test, 28/28 E2E pass
27	TenantGuard for JWT-based tenant isolation
28	Case lifecycle E2E + GDPR export fix
29	Kafka chaos E2E resilience tests
30	Analytics & Decision query E2E tests
31	Health check E2E for all 13 microservices

Sprint	Key Deliverables
32	Analytics consolidation, health checks, merchant CRUD
33	Feature flags E2E, DLQ timeout fix (5s), test stability (78/78)
34	Kafka timeout resilience (producer 10s, lag poll 10s), deferred poll, test stability
35	Stateful fraud gap closure: orchestrator→fetchAllSignals()+DSL live evaluation, typed entity prior-decision memory (migrations 013-014), feature registry parity

For production deployment details see `docs/runbooks/go-live-checklist.md`*. For security incident response see* `docs/runbooks/on-call-playbook.md`*.*